

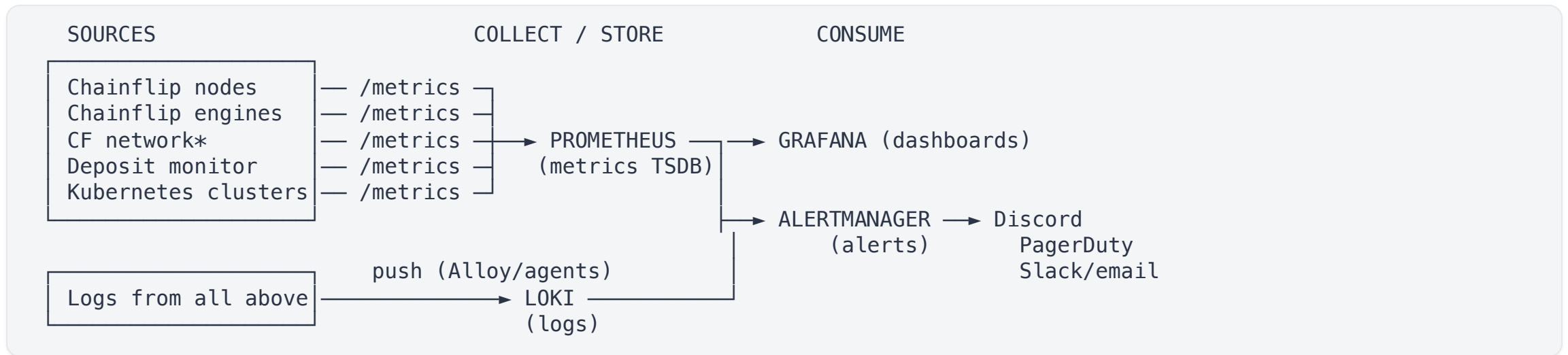
Monitoring & Observability at Chainflip

How we know what the network is doing

Two pillars: Prometheus (metrics) · Loki (logs)

On top: Grafana (dashboards) · Alertmanager (alerts)

The big picture



* the Chainflip **network** is observed via our **chainflip-prometheus-exporter**

Prometheus — what it is

- Open-source **metrics monitoring** system + **time-series database**.
- **Pull-based:** Prometheus *scrapes* an HTTP `/metrics` endpoint on each target every `scrape_interval`.
- Each target just exposes its current values as plain text in the **Prometheus exposition format** — one `metric_name{labels} value` line per metric; Prometheus does the collecting & storing.

Prometheus — data model & PromQL

- A **time series** = a metric name + a set of key/value **labels**.

```
cf_block_height{chain="ethereum", network="mainnet"}
```

- A **sample** = `(timestamp, value)`.
- Changing *any* label value creates a **new, distinct** series.
-  **Cardinality matters:** never put unbounded values (user IDs, tx hashes) in labels — every combination is its own series and memory explodes.
- **PromQL** — Prometheus's functional query language for selecting & aggregating these series in real time, e.g. `rate(cf_events_count_total[5m])` or `sum by (chain) (...)`.
- **Storage:** local on-disk TSDB with short retention; long-term → remote-write to **VictoriaMetrics**.

How our sources expose metrics

- **Chainflip nodes & engines** — expose their own `/metrics` directly.
- **Kubernetes** — `node_exporter` (host) + `kube-state-metrics` (cluster objects); Prometheus auto-discovers pods via k8s service discovery.
- **Deposit monitor** — instrumented service exposing `/metrics`.
- **The Chainflip network itself** has no `/metrics` — on-chain state isn't an HTTP target. → that's the job of our **chainflip-prometheus-exporter**.

Our component: the chainflip-prometheus-exporter

- A **Node/TypeScript** service that reads **on-chain data & state**, turns it into Prometheus metrics.
- **8 chain watchers**, each with its own registry, individually toggled per network (localnet / perseverance / berghain / mainnet):

Chainflip State Chain · Ethereum · Bitcoin · Arbitrum · BSC · Solana · AssetHub · Tron

- Also exposes `/health/block-lag` probes that flag when an external chain's witnessing falls behind its real chain tip.

What the exporter measures

State Chain (`cf_*`) — the richest source:

- Chain health: `cf_block_height`, `cf_external_chain_block_height{tracked_chain=...}`, reorg detection
- Validators/epoch: `cf_authorities`, `cf_authorities_online`, `cf_suspended_validators`, `cf_reputation`
- Operations: rotations, `cf_swapping_queue`, `cf_open_deposit_channels`, `cf_pending_redemptions`, broadcasts
- Economics: `cf_flip_total_supply`, fees, lending/LTV (`cf_ltv_ratio_histogram`), oracle prices

External chains: block height + balances (`eth_balance`, `eth_token_balance`, `bsc_bnb_balance`, `bsc_token_balance`, `sol_balance`, `tron_trx_balance`)

Code structure

```
src/  
├─ app.ts      # entry  
├─ watchers/   # one service per chain – the polling / subscription loop  
├─ metrics/    # metric definitions, grouped by chain (chainflip, eth, btc, sol, ...)  
├─ config/     # load & validate per-network settings  
...
```

Each file in `metrics/<chain>/` does two things — **define** the metric (name, help, type) and **transform** on-chain data into a Prometheus sample. The chain's **watcher** calls it on every new block / poll with freshly-read `data`.

➡ **Adding a metric** = define it in `metrics/<chain>/`, then populate it from that chain's `watchers/<chain>.ts`.

Loki — logs, the Prometheus way

- "Like Prometheus, but for logs"
- **Indexes only labels** (service, namespace, pod...), not full log text → raw lines. Cheap & simple vs full-text systems like Elasticsearch.
- **Push-based:** agents ship logs to Loki — **Promtail**
- **LogQL** mirrors PromQL: same `{label="value"}` selectors, can turn logs into metrics (`rate`, `count_over_time`).
- Loki's **ruler** can alert on logs and sends to the **same Alertmanager**.

Alerting — how an alert is born (Prometheus side)

1. We **define alerting rules** = PromQL expressions with labels + annotations.

```
- alert: EthBlockLagHigh
  expr: (eth_block_height - on(...) cf_external_chain_block_height{tracked_chain="ethereum"}) > 300
  for: 5m
  labels: { severity: critical }
  annotations: { summary: "ETH witnessing lagging" }
```

2. Every **evaluation_interval** Prometheus evaluates the expression.
3. Condition true → alert is **pending**; stays true for the **for** duration → **firing**. (*This **for** clause kills transient spikes / flapping.*)
4. Prometheus **pushes** firing alerts to **Alertmanager** (and keeps re-sending while active).

Alertmanager — dedup, route, notify

Prometheus decides *what* is wrong; **Alertmanager decides who hears about it and how**. Five jobs:

- **Grouping** — batch related alerts into one notification (`group_by`).
- **Deduplication** — collapse identical alerts
- **Inhibition** — suppress noise when a bigger alert fires (mute per-node alerts when the whole cluster is down).
- **Silencing** — time-boxed manual muting (maintenance windows).
- **Routing** — a **routing tree** matches alert labels → a **receiver**.

Receivers: Discord (which channel via the route), **PagerDuty (PD)**, webhooks.

Where to find things

Want to extend the monitoring yourself? Three repos:

- `chainflip-prometheus-exporter` — to add **metrics** about the Chainflip network or external-chain-specific things (e.g. Solana durable nonces, a new balance, a new chain watcher).
- `chainflip-alerting` — to add or tune **alerts** (the PromQL/LogQL rules)
- `chainflip-cluster-configuration` — to update **Alertmanager** config & **routing** (which receiver / Discord channel / PagerDuty an alert lands in).

| Questions?